CHAPTER 06

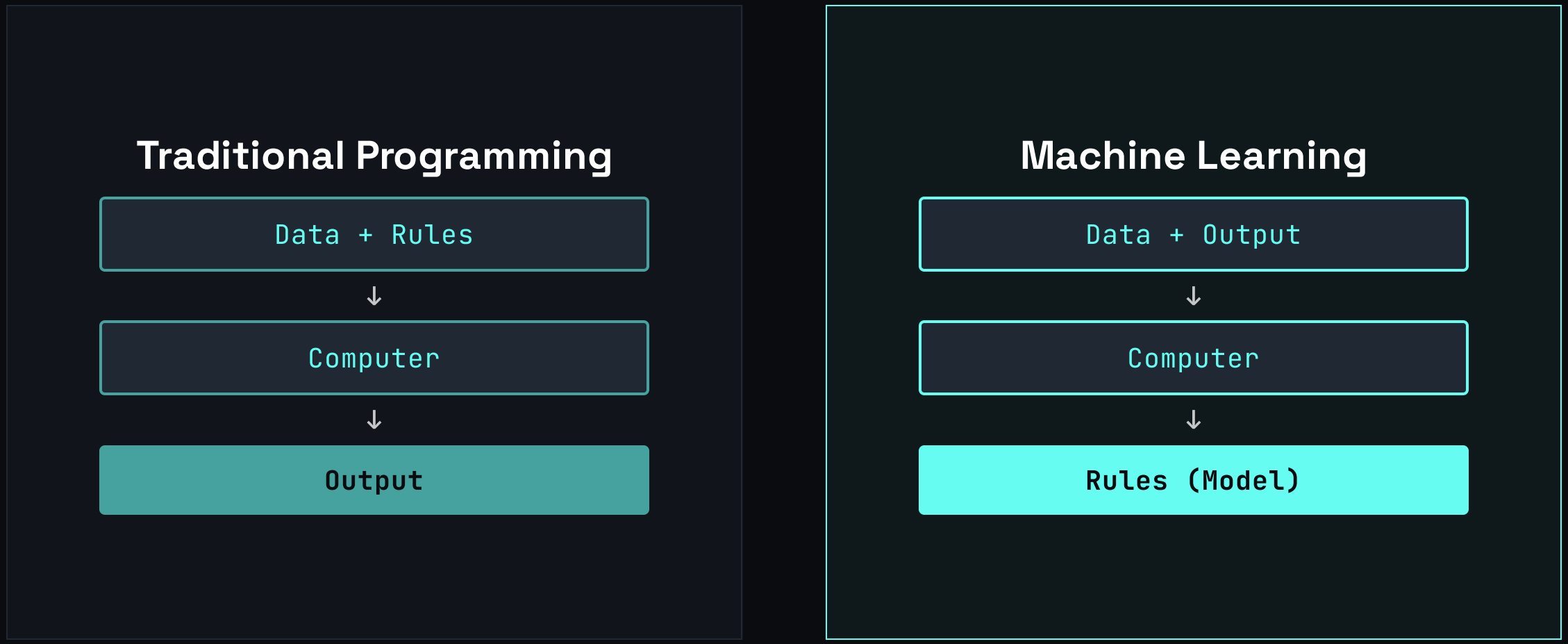
Introduction to Machine Learning

From Data to Decisions

CAI1001C | Artificial Intelligence Thinking

Traditional Programming vs. Machine Learning

A FUNDAMENTAL SHIFT IN THINKING



We don’t program the rules. We program the system to **learn** the rules.

Your Mission for This Chapter

KEY LEARNING OBJECTIVES

01

Define Machine Learning

Understand the shift from rules to data and the three main paradigms.

02

Master the Workflow

Learn the end-to-end process: from problem definition to deployment.

03

Build Your First Model

Implement Linear Regression using Python and scikit-learn.

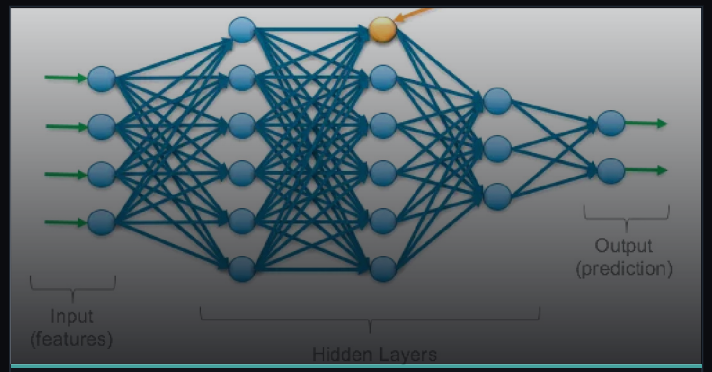
04

Evaluate Performance

Go beyond "it works" to understand metrics like R^2 and MSE.

How Machines Learn

THREE PARADIGMS



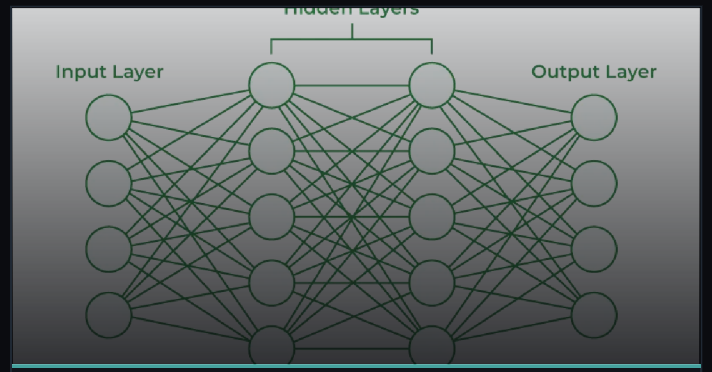
Supervised

Labeled Data

The model learns from examples with known answers (a "teacher").

USE CASES

Spam Detection, Price Prediction, Medical Diagnosis



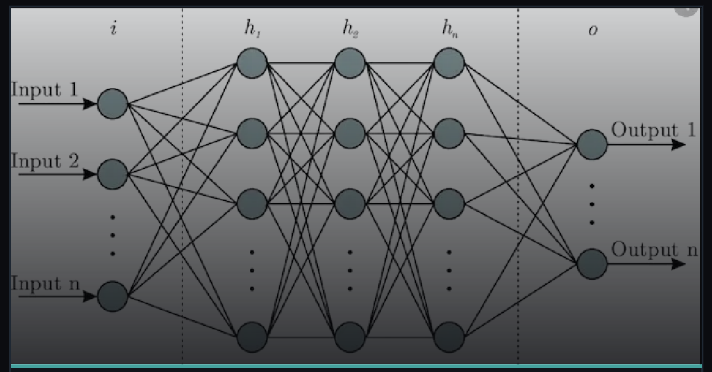
Unsupervised

Unlabeled Data

The model finds hidden patterns or structures in data on its own.

USE CASES

Customer Segmentation, Anomaly Detection, Recommendation



Reinforcement

Rewards & Penalties

The model learns by trial and error in an interactive environment.

USE CASES

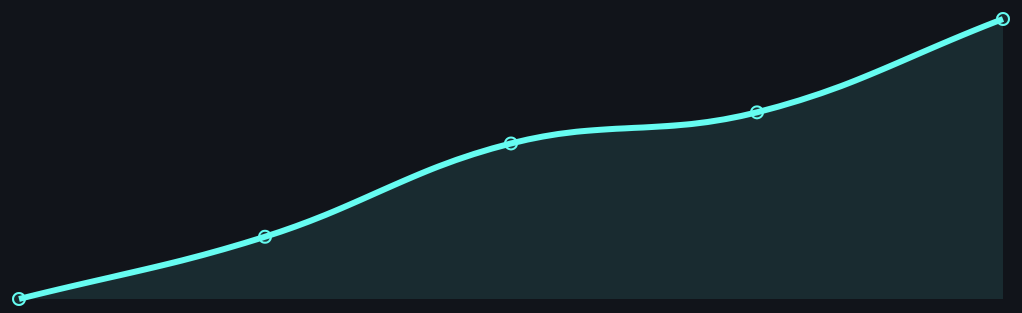
Game AI, Robotics, Self-Driving Cars

Supervised Learning

THE TWO MAIN TYPES

Regression

Predicting a Number

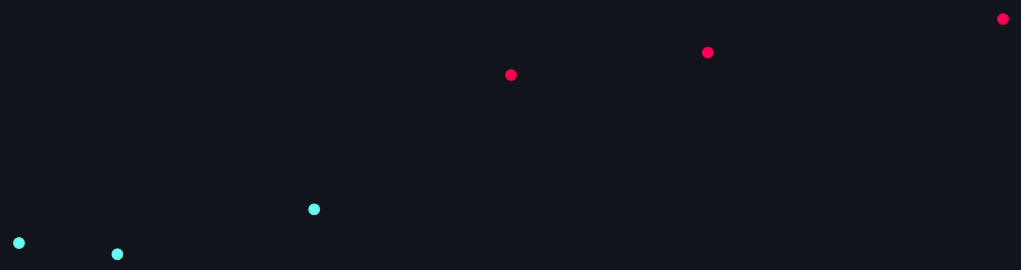


EXAMPLE QUESTION

"How much will this house sell for?"

Classification

Predicting a Category



EXAMPLE QUESTION

"Is this email Spam or Not Spam?"

Both types require historical labeled data to learn patterns.

The Machine Learning Workflow

AN END-TO-END PROCESS

01

Define Problem

What are we predicting? Identify features (inputs) and label (output).

→

02

Collect Data

Gather historical records with known outcomes to learn from.

→

03

Prepare Data

Clean, format, and handle missing values (Garbage In, Garbage Out).

↓

04

Split Data

Separate into Training Set (80%) and Test Set (20%) to prevent cheating.

→

05

Train Model

Feed training data to the algorithm so it can find the patterns.

→

06

Evaluate

Test on unseen data. If good, deploy. If bad, iterate.

Translating Problems into ML Terms

SPEAKING THE LANGUAGE OF DATA

Features (X)

The **inputs.** The information the model uses to make a decision. Also called predictors or variables.

Label (y)

The **output.** The answer you want the model to predict. Also called the target.

Given **[Features]**,
Predict **[Label]**

EXAMPLE: E-COMMERCE PREDICTION

User ID	Time on Site	Pages Viewed	Device	Purchased?
1001	45 sec	2	Mobile	No
1002	320 sec	8	Desktop	Yes
1003	120 sec	4	Mobile	No
1004	540 sec	12	Desktop	Yes
1005	15 sec	1	Tablet	No

Features (Inputs) Label (Output)

The Golden Rule: The Train/Test Split

PREVENTING MEMORIZATION, ENSURING LEARNING

Training Set (80%) | Test Set (20%)

TRAIN | TEST

The Study Session

Like studying with practice problems. The model sees both the **Questions (Features)** and the **Answers (Labels)**. It learns the patterns.

The Final Exam

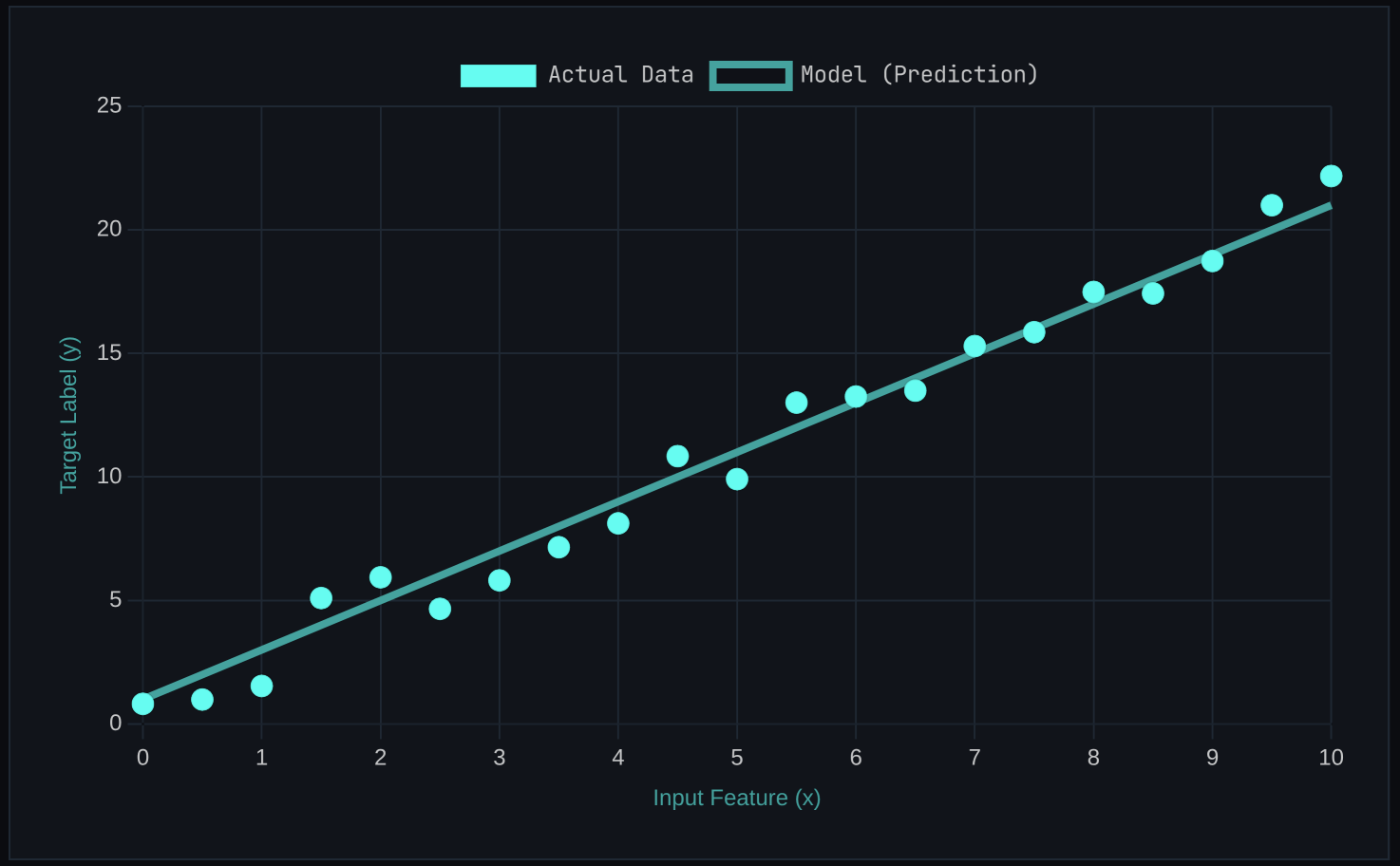
Like taking the test. The model sees **ONLY Questions.** It must predict the answers. We grade it by comparing its predictions to the real answers we hid.

Real-world application: AI interacting with new, unseen data.

Critical: If you test on the data you trained on, the model is just memorizing. That's cheating.

Finding the "Line of Best Fit"

LINEAR REGRESSION



The Goal

Draw a straight line through the data points that minimizes the total distance (error) between the points and the line.

The Prediction

Once we have the line, we can predict the output (y) for any new input (x) just by finding where it falls on the line.

THE MATH MODEL

$$y = mx + b$$

From Concept to Code

IMPLEMENTING LINEAR REGRESSION

model_training.py

```python
# 1. Import the model class
from sklearn.linear_model import LinearRegression

# 2. Initialize the model
model = LinearRegression()

# 3. Train the model (The "Learning" Step)
model.fit(X_train, y_train)

# 4. Make predictions on new data
```

Measuring Performance

HOW GOOD IS THE MODEL?

R-Squared (R^2)

"The Fit Score." Represents the proportion of variance in the output explained by the input.

Range: 0 to 1 (0% to 100%)
Goal: Higher is Better

0.85

STRONG FIT

MSE

"The Error Score." Mean Squared Error. The average squared difference between predicted and actual values.

Range: 0 to ∞
Goal: Lower is Better

12.4

AVERAGE ERROR

REAL WORLD CONTEXT

Dashboard 1

Dashboard 2

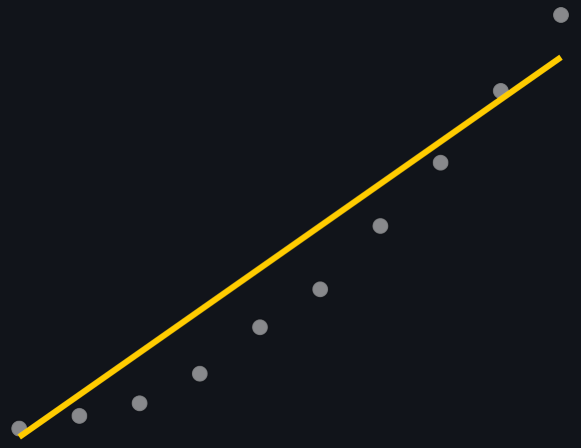
Dashboard 3

When Memorization Goes Wrong

THE GOLDILOCKS PROBLEM

Underfitting

TOO SIMPLE



The model is too rigid. It fails to capture the underlying pattern. Like studying only the first page of the textbook.

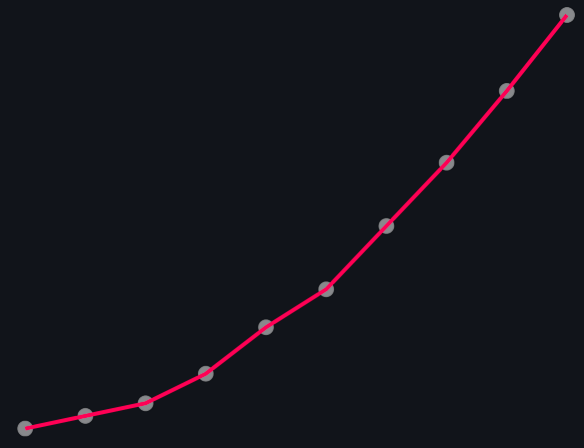
Good Fit

JUST RIGHT

The model captures the signal but ignores the noise. It generalizes well to new, unseen data.

Overfitting

TOO COMPLEX

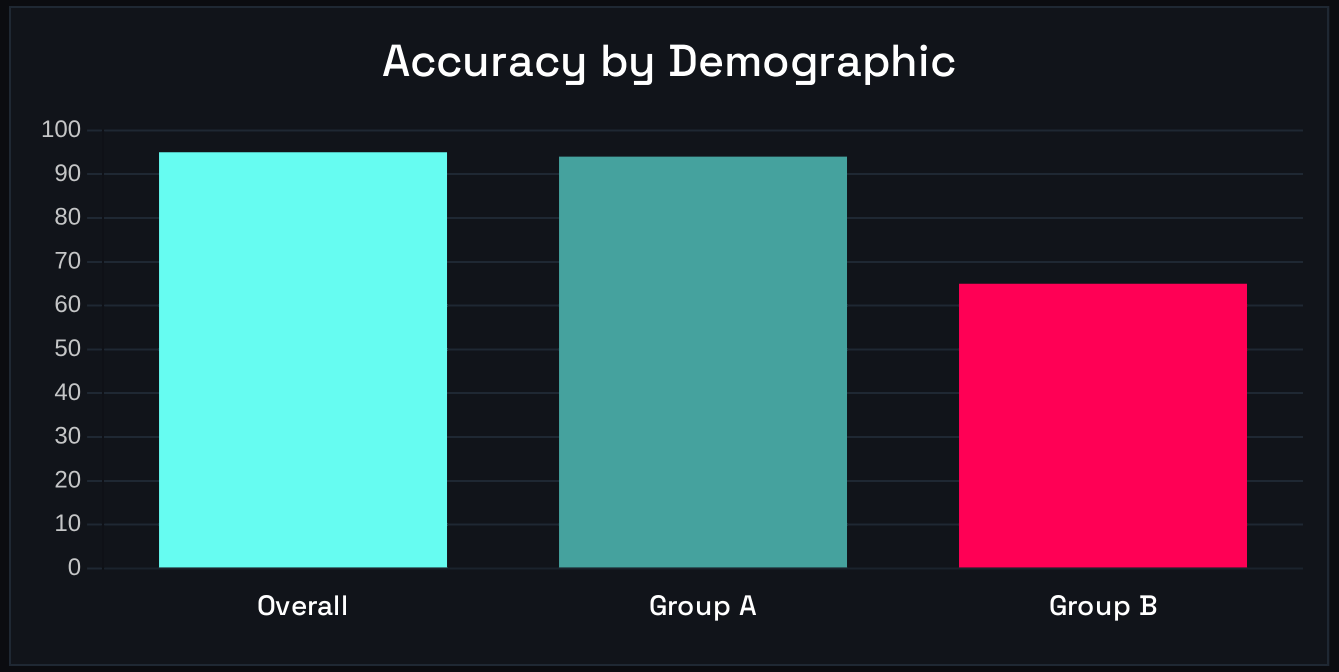


The model memorizes the noise. It hits every training point but fails miserably on the test. Like memorizing the answer key.

Goal: **Generalization**. We want the model to learn the concept, not the specific examples.

A Critical Check: Is the Model Fair?

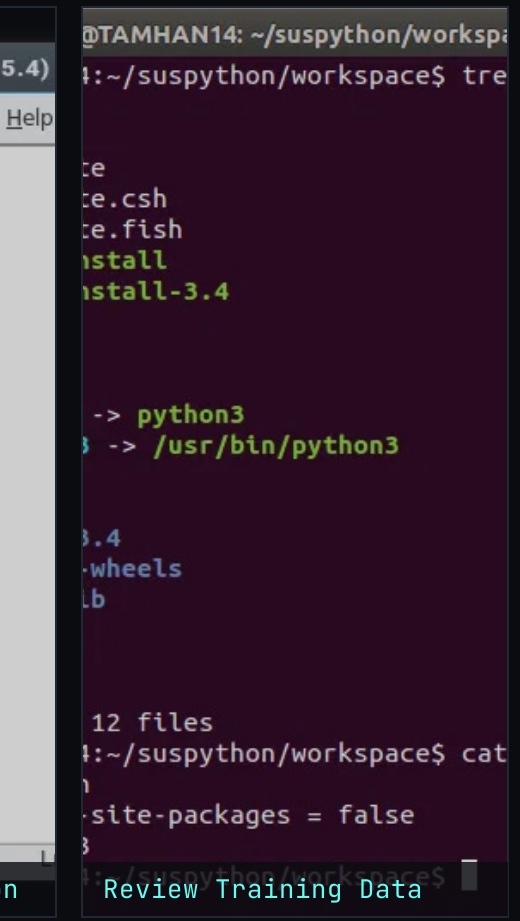
BEYOND OVERALL ACCURACY



Warning: The model looks great overall (95%), but fails significantly for Group B (65%). This is **Algorithmic Bias.**

TECHNICAL AUDIT

Check Feature Selection



Review Training Data

Mission Debrief

KEY TAKEAWAYS

The Paradigm Shift

We moved from programming explicit rules to programming systems that **learn rules from data.**

The Golden Rule

Never test on training data. The 80/20 split is the only way to measure true learning vs. memorization.

Linear Regression

A supervised learning algorithm that finds the **line of best fit** to predict continuous numerical values.

Responsible AI

Accuracy isn't enough. We must check for **overfitting** and ensure our models are **fair** to all groups.

NEXT MODULE LOADING...

Chapter 7: Classification